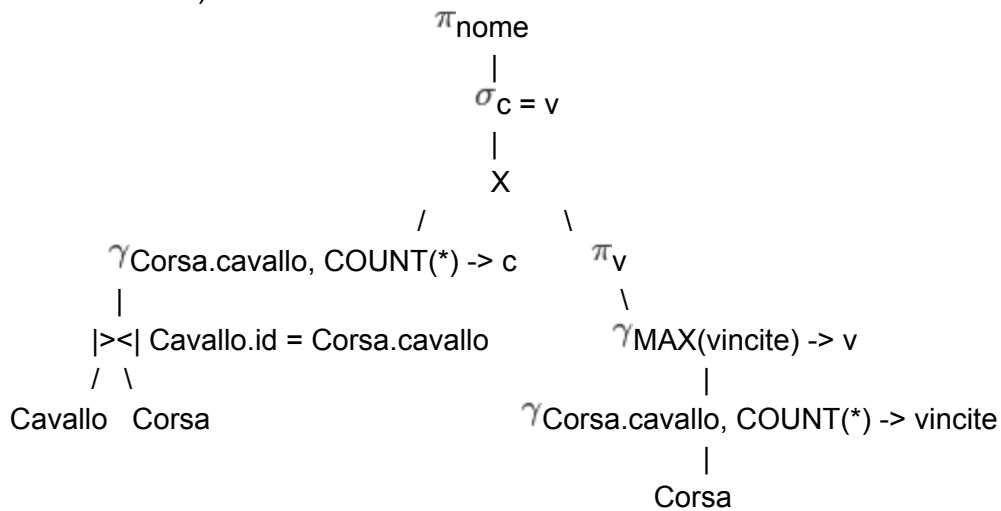


ESAME BASI DI DATI 18/01/2011

1)

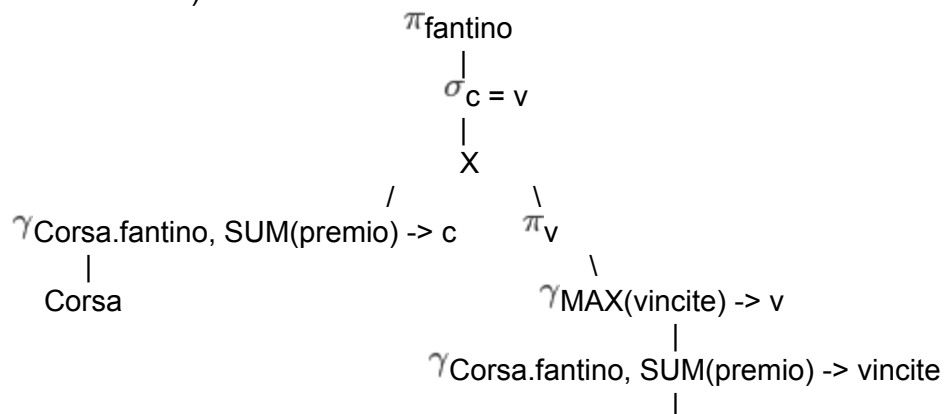
a.

```
SELECT nome
FROM Cavallo, Corsa
WHERE Cavallo.id = Corsa.cavallo
GROUP BY Corsa.cavallo
HAVING COUNT(*) = (SELECT MAX(vincite)
                    FROM (SELECT COUNT(*) AS vincite
                          FROM Corsa
                          GROUP BY Corsa.cavallo)
                    )
```



b.

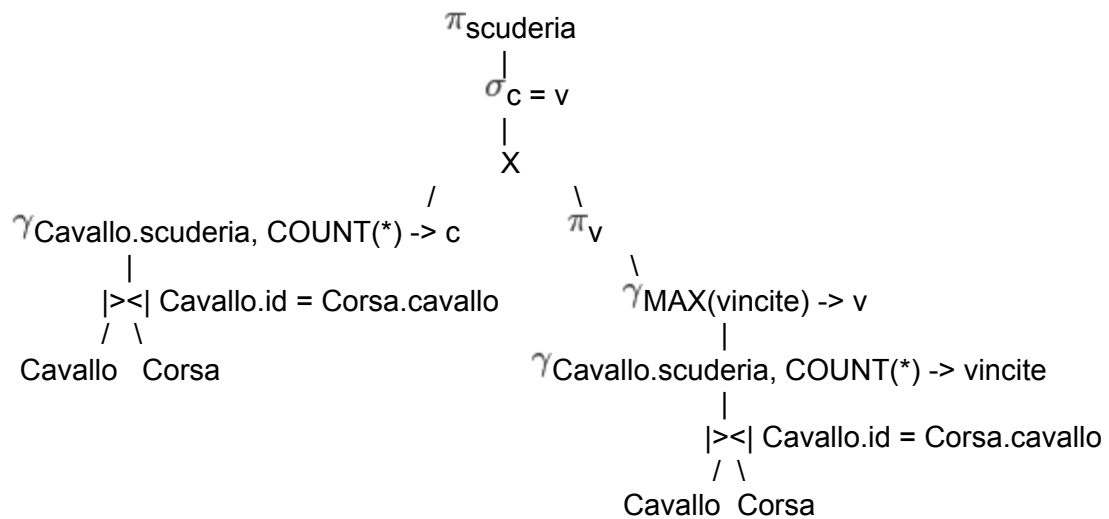
```
SELECT fantino
FROM Corsa
GROUP BY Corsa.fantino
HAVING SUM(premio) = (SELECT MAX(vincite)
                      FROM (SELECT SUM(premio) AS vincite
                            FROM Corsa
                            GROUP BY Corsa.fantino)
                      )
```



Corsa

c.

```
SELECT scuderia
FROM Cavallo, Corsa
WHERE Cavallo.id = Corsa.cavallo
GROUP BY Cavallo.scuderia
HAVING COUNT(*) = (SELECT MAX(vincite)
                    FROM (SELECT COUNT(*) as vincite)
                        FROM Cavallo, Corsa
                        WHERE Cavallo.id = Corsa.cavallo
                        GROUP BY Cavallo.scuderia)
)
```



d.

```
SELECT fantino
FROM Corsa
GROUP BY Corsa.fantino
HAVING COUNT(DISTINCT cavallo) = (SELECT MAX(c)
                                   FROM (SELECT COUNT(DISTINCT cavallo) AS c
                                         FROM Corsa
                                         GROUP BY Corsa.fantino)
                                   )
)
```

2.

RIVISTA (nome)

| 1..1
|

1..n	1..1	1..1
ARTICOLO (titolo, anno_pubbl, numero pagine)	-----	ARGOMENTO (desc_breve, desc_lunga, materia)
1..n		
1..n		
RICERCATORE (nome, cognome, cf)		

3.

r4(E) r1(A) w1(D) w2(B) w1(C) w4(C) r1(E) w2(A) r3(D) w3(A) r4(B)

Vincoli:

attributo A) 1 viene prima di 2 per r1(A) e w2(A)
 1 viene prima di 3 per r1(A) e w3(A)
 2 viene prima di 3 per w2(A) e w3(A)
 attributo B) 2 viene prima di 4 per w2(B) e r4(B)
 attributo C) 1 viene prima di 4 per w1(C) e w4(C)
 attributo D) 1 viene prima di 3 per w1(D) e r3(D)
 attributo E) nessuna info

GRAFO: / \
 1 -----> 2 -----> 4
 | /
 v /
 3 <

Non ci sono cicli quindi è conflict serializable.

Esecuzioni seriali equivalenti:

T1 T2 T3 T4

T1 T2 T4 T3

4.

a.

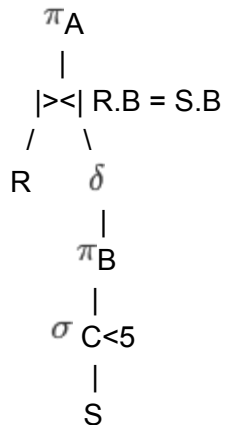
<Q>

/					\
SELECT	<sellist>	FROM	<fromlist>	WHERE	<condition>
	<attribute>		<relname>		<attribute> IN <query>
	A		R		B

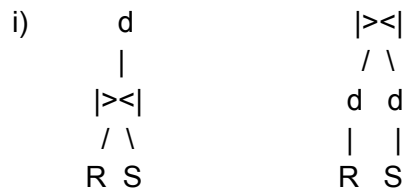
SELECT <sellist> FROM <fromlist> WHERE <condition>

	<attribute>		<relname>		<attribute> <
	B		S		C

5



b.



Nel primo caso abbiamo un costo che è solo quello del join: $1000 \cdot 200 / 60 = 3334$

Nel secondo caso abbiamo i due distinct: $1000/2 + 200/2 = 500 + 100 = 600$

Meglio fare prima il distinct

ii) primo caso: $1000 \cdot 200 / 1000 = 200$

secondo caso: 600

Conviene fare prima il join

c.	T	V(B)	V(C)
R	1000	6	
S	2000	40	40
U	5000		50
RS -> $1000 \cdot 2000 / 40 = 50000$			
RU -> $1000 \cdot 5000 = 5000000$			
SU -> $2000 \cdot 5000 / 50 = 200000$			

Conta solo il piano intermedio per vedere qual è il migliore, scelgo il migliore tra questi
RSU è il piano migliore

d.

sigma A=1 AND B=2

Ogni blocco contiene in media $1000/100 = 10$ tuple

Per $A=1$ stimo $1000/50 = 20$ tuple

Per $B=2$ stimo $1000/200 = 5$ tuple

Possibilità:

- Dato che ho un indice clustering su A lo uso e $\sigma(A=1)$ mi costa 2 (stimo di dover prendere due blocchi dal disco dato che devo prendere 20 tuple). Tra queste 20 tuple "filtro" quelle con $B!=2$ e ottengo il risultato.

COSTO: 2

- Su B ho un indice non clustering quindi se lo uso $\sigma(B=2)$ mi costa 5 (nel peggiore dei casi le 5 tuple saranno in 5 blocchi diversi). Tra queste "filtro" quelle con $A!=1$.

COSTO: 5

- Non uso nessun indice. Devo fare un table-scan completo, quindi leggere tutti i blocchi.

COSTO: $B(R)$ (= 100)

Quindi mi conviene usare l'indice clustering su A